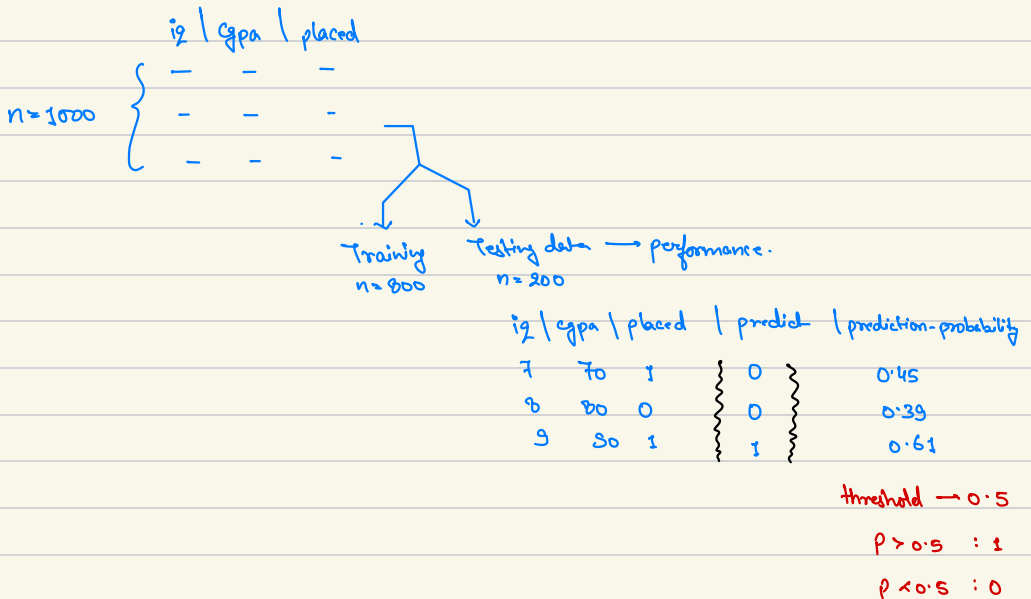




# ROC Curve in ML

Why ROC Curve?

1. Threshold Selection  $\rightarrow$  Related to Classification problem. [Binary Classification Problem]



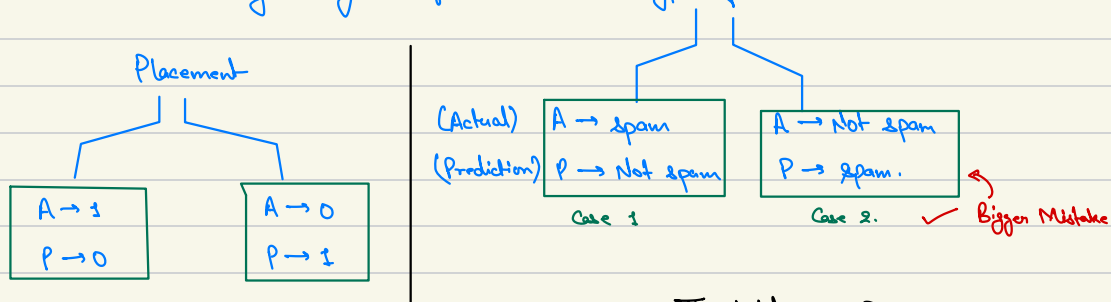
In case of prediction we do not get the 0/1 directly, instead we get the prediction-probability instead. Now, we decide a threshold (e.g. 0.5)  
if  $P > 0.5$  then 1 (placed) else 0.

Not always the threshold (0.5) number is correct.

Eg) E-mail spam classification.

$\rightarrow$  email  $\rightarrow$  new email  
 $\downarrow$   
spam | Not spam.

In solving Binary Classification  $\rightarrow$  2 types of mistakes.



Threshold  $\rightarrow 0.5$

$\downarrow$   
 Not spam  $\leftarrow$  0.75  $\rightarrow$  Spam

With the help of threshold we are able to reduce a particular kind of error.

This is called **threshold selection**.

Still the threshold is unknown.

To make an educated decision we use **ROC curve**.

Confusion Matrix.

|        |   | Predicted      |                |
|--------|---|----------------|----------------|
|        |   | 1              | 0              |
| Actual | 1 | True Positive  | False Negative |
|        | 0 | False Positive | True Negative  |

| i2 | cypa | placd | predicted |
|----|------|-------|-----------|
| 70 | 7    | 1     | 1         |
| 80 | 8    | 0     | 1         |
| 90 | 9    | 1     | 0         |

|   |   | P   |    |
|---|---|-----|----|
|   |   | 1   | 0  |
| A | 1 | 100 | 5  |
|   | 0 | 15  | 80 |

Correct Prediction  $\rightarrow 100 + 80 = 180$

Wrong Prediction  $\rightarrow 15 + 5 = 20$

True Positive Rate :  $\rightarrow$  Benefit.

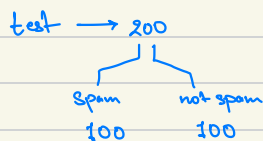
|   |   | P   |     |
|---|---|-----|-----|
|   |   | 1   | 0   |
| A | 1 | T.P | F.N |
|   | 0 | F.P | T.N |

$$\text{T.P.R} = \frac{\text{T.P}}{\text{T.P} + \text{F.N}}$$

Eg.) Email spam classifier.

1  $\rightarrow$  spam.

0  $\rightarrow$  Not spam



|   |   | P  |    |
|---|---|----|----|
|   |   | 1  | 0  |
| A | 1 | 80 | 20 |
|   | 0 | 20 | 80 |

$$\text{T.P.R} = \frac{80}{80 + 20} = 0.80$$

Out of 100 spam emails, we were able to identify 80 spam mails.

T.P.R  $\rightarrow$  gives intuition of benefit.

Churn Rate Prediction :

|  |   | 1  | 0  |
|--|---|----|----|
|  | 1 | 80 | 20 |
|  | 0 |    |    |

T.P.R  $\uparrow$  good thing.

1  $\rightarrow$  100%      0  $\rightarrow$  T.P = 0  
F.N = 0

T.P.R = 0.8 [80% churn was predicted]

False Positive Rate :  $\rightarrow$  Cost.

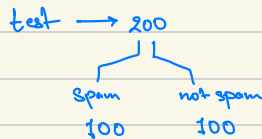
|   |   | P   |     |
|---|---|-----|-----|
|   |   | 1   | 0   |
| A | 1 | T.P | F.N |
|   | 0 | F.P | T.N |

$$F.P.R = \frac{F.P}{F.P + T.N}$$

Eg.) Email spam classifier.

1  $\rightarrow$  spam.

0  $\rightarrow$  Not spam



|   |   | P  |    |
|---|---|----|----|
|   |   | 1  | 0  |
| A | 1 | 80 | 20 |
|   | 0 | 20 | 80 |

$$F.P.R = \frac{20}{100} = 0.2.$$

not spam  $\xrightarrow[\text{as}]{\text{classified}}$  Predicted spam

Not leaving the platform  $\rightarrow$  they will leave.

F.P.R  $\rightarrow$  Cost.

When  $F.P.R = 1$  (It's worst condition)

$$T.N = 0$$

When  $F.P.R = 0$ ,  $F.P = 0$

Best Case :

$$T.P.R = 100\% \text{ or } 1 \rightarrow F.N = 0$$

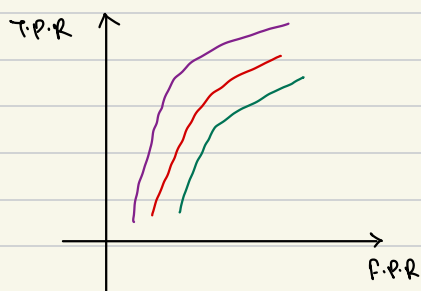
$$F.P.R = 0\% \text{ or } 0 \rightarrow F.P = 0$$

|     |     |
|-----|-----|
| 100 | 0   |
| 0   | 100 |

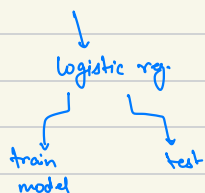
## ROC Curve :

ROC  $\rightarrow$  Receiver Operator Characteristic.

Graph plotted b/w T.P.R and F.P.R is called ROC Curve.



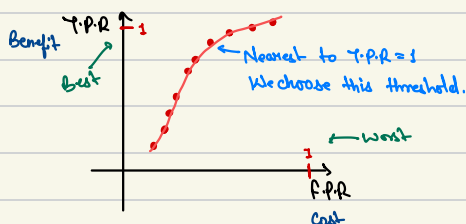
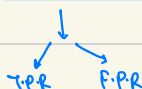
eggs / ig / placed



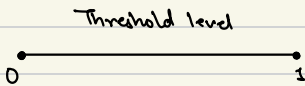
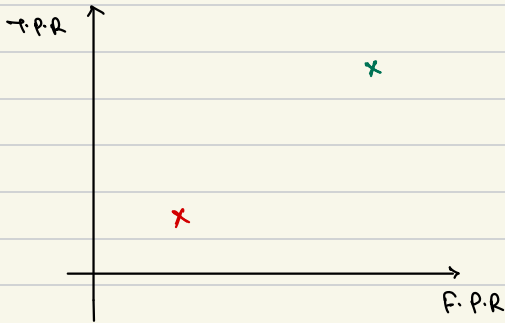
threshold  $\rightarrow$

0.5, 0.6, 0.7, . . . .

|     |     |
|-----|-----|
| T.P | FN  |
| F.P | T.N |



## Different Cases :



① Small  $\rightarrow 0.1 \cong 0$

② Large  $\rightarrow 0.95 \cong 1$

Eg. Spam Classifier.

Case 1: threshold  $\rightarrow 0.1 \cong 0$

mails  $\rightarrow$  spam.

$P(0.2) \rightarrow$  spam.

$P < 0.1 \rightarrow$  not spam.  
400s

|     | P              |                  |
|-----|----------------|------------------|
|     | 1              | 0                |
| A 1 | T.P $\uparrow$ | F.N $\downarrow$ |
| 0   | F.P $\uparrow$ | T.N $\downarrow$ |

$\therefore$  T.P.R  $\uparrow \rightarrow$  Decreased Threshold.  
 $\therefore$  F.P.R  $\uparrow$

Case 2: threshold  $\rightarrow 0.95$

$P > 0.95$  : Spam

$P < 0.95$  : Not spam.

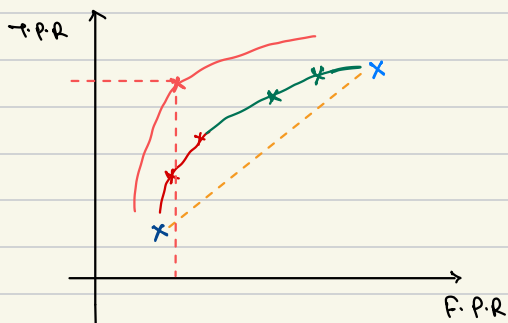
|     | P                |                |
|-----|------------------|----------------|
|     | 1                | 0              |
| A 1 | T.P $\downarrow$ | F.N $\uparrow$ |
| 0   | F.P $\downarrow$ | T.N $\uparrow$ |

$\therefore$  T.P.R  $\downarrow \rightarrow$  Increased Threshold.  
 $\therefore$  F.P.R  $\downarrow$

|   | Predicted. |     | Actual |
|---|------------|-----|--------|
|   | 1          | 0   |        |
| 1 | T.P        | F.N |        |
| 0 | F.P        | T.N |        |

$$T.P.R = \frac{T.P}{T.P + F.N}$$

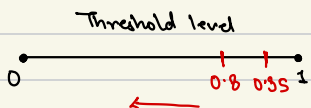
$$F.P.R = \frac{F.P}{F.P + T.N}$$



|   | 1          | 0   |        |
|---|------------|-----|--------|
| 1 | T.P        | F.N | Actual |
| 0 | F.P        | T.N |        |
|   | Predicted. |     |        |

$$T.P.R = \frac{T.P}{T.P + F.N}$$

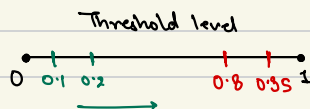
$$F.P.R = \frac{F.P}{F.P + T.N}$$



Threshold = 0.8

$P > 0.8$  : Spam.

$P < 0.8$  : Not spam.



|      |       |
|------|-------|
| TP ↑ | F.N ↓ |
| F.P  | T.N   |

$\therefore T.P.R \uparrow$

$\therefore F.P.R$  [Not much change]

## AUC ROC Curve :

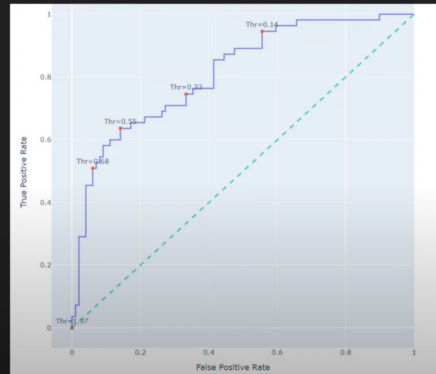
### AUC-ROC

05 June 2023 12:49

The AUC-ROC measures the entire two-dimensional area underneath the entire ROC curve from (0,0) to (1,1). AUC provides an aggregate measure of performance across all possible classification thresholds.

- An AUC of 1 indicates that the model has perfect discrimination: it correctly classifies all positive and negative instances.
- An AUC of 0.5 suggests the model has no discrimination ability: it is as good as random guessing.
- An AUC of 0 indicates that the model is perfectly wrong: it classifies all positive instances as negative and all negative instances as positive.

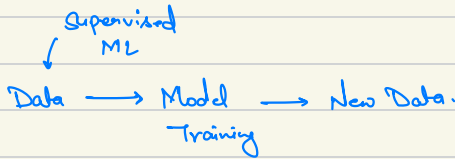
In practice, AUC values usually fall between 0.5 (random) and 1 (perfect), with higher values indicating better classification performance.



AUC  $\rightarrow$  Area Under Curve.

# Cross Validation

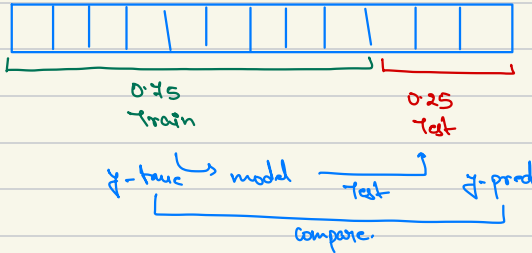
The Problem :  
(Why ?)



Cross Validation  $\rightarrow$  Train a model & evaluate it using certain technique.

The Hold Out Approach :  
 $\hookrightarrow$  Train-test-split

1000 Customers.



Cross Validation : It's better than Hold-out Approach.

Problems with The Hold-Out Approach :

1. Variability  $\rightarrow$  Performance of model is very sensitive to how data is divided.

random\_state = 1  $\rightarrow$  1st accuracy = 90%.

" " 2  $\rightarrow$  accuracy changes.

So, it's not reliable. Due to high variability.

2. Data inefficiency  $\rightarrow$  It uses a portion of the data for training & a different portion for testing.
3. Bias in performance estimation  $\rightarrow$  Bias may reduce or increase due to over or under represented in training data.
4. Less reliable for hyperparameter tuning.

Why is Hold-out Approach used then?

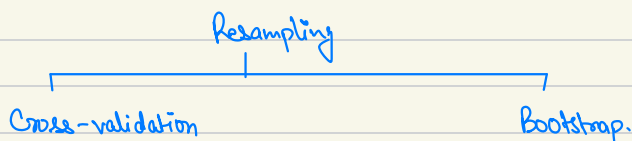
1. Simplicity
2. Computational Efficiency
3. Large Datasets.  $\rightarrow$  For large datasets, even a small portion of data may be sufficient to form a representative test set.

Cross-Validation :

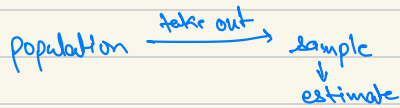
The idea of CV is to divide the data into several subsets or "folds".

Model is trained on some of these subsets and tested, on the remaining ones.

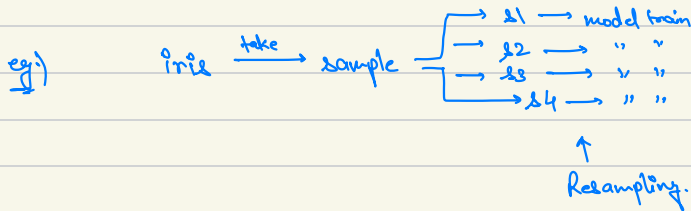
The process is repeated multiple times, with different subsets used for training and validating each time. The results from each rounds are usually averaged to estimate model's overall performance.



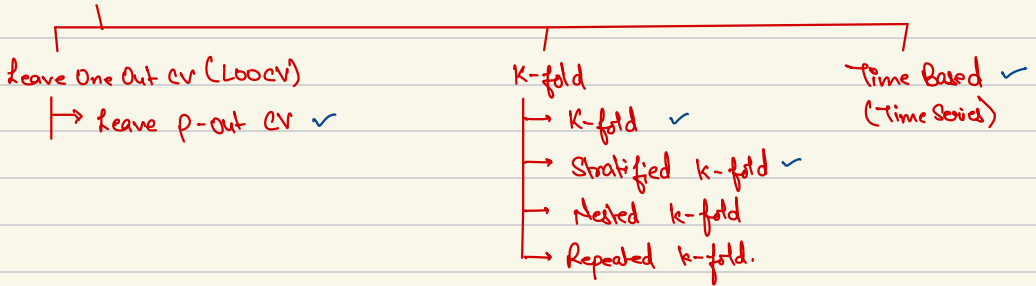
Resampling  $\rightarrow$  sampling



Resampling : creation of new sample based on one observed sample.



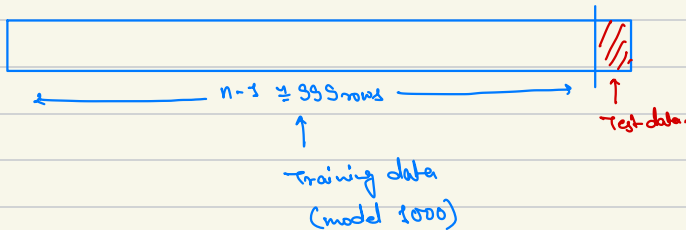
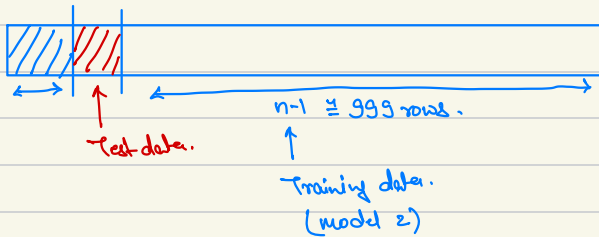
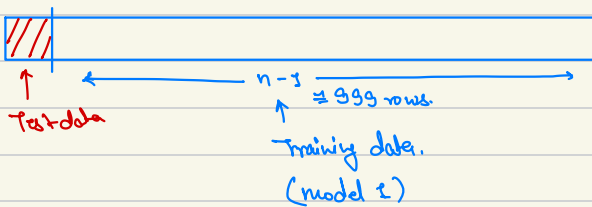
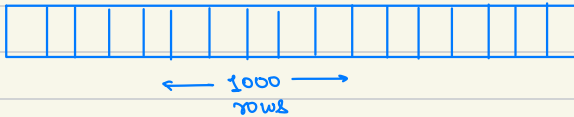
Cross Validation



$\checkmark \rightarrow$  important asked in interviews.

## Leave One Out Cross Validation (LOOCV) :

Dataset  $\rightarrow$  1000 rows.  $\rightarrow$  models  $\rightarrow$  evaluation.  
(n) (1000)



Accuracy :  $m_1 + m_2 + m_3 + \dots + m_{1000} \rightarrow \text{avg} \rightarrow \text{Final Accuracy Score}$   
(LOOCV)

### Advantages :

1. Use of full data.
2. Less Bias.
3. No randomness.

### Dis-advantages :

1. Computational Expensive.
2. High variance.
3. Inappropriate Performance Metric
4. Not ideal for imbalanced data.

### When to use :

1. Small dataset.
2. Balanced datasets.
3. Need for less biased performance estimate.

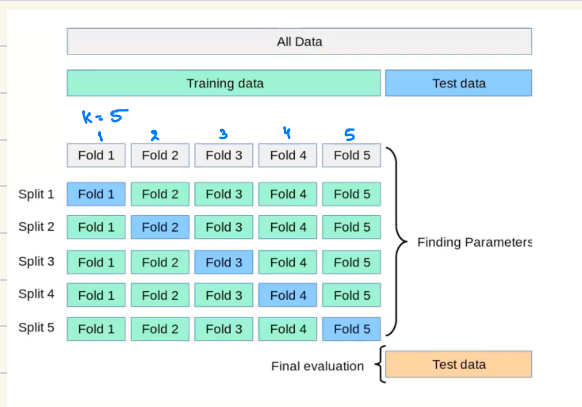
## K-Fold Cross Validation :

\* Most used cross-validation technique.

\* Empirically we take value of  $k = 5$  or  $10$ .

$k=5 : n=3500 <$

$k=10 : n=3500 >$



### Advantages :

1. Reduction of Variance.
2. Computationally inexpensive.

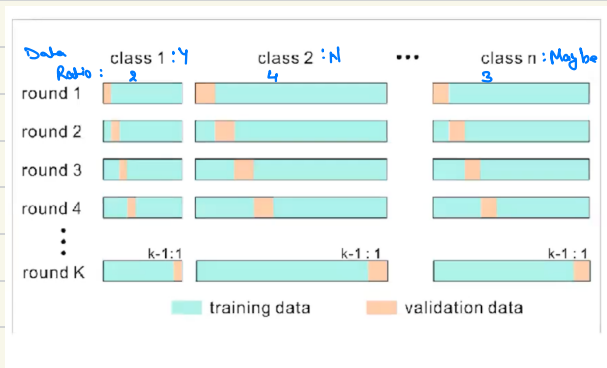
### Disadvantages :

1. Potential for High Bias.
2. May not work well with imbalanced data.

### When to use ?

1. Sufficiently Large Datasets.
2. Data is evenly distributed.

## Stratified K-Fold CV :



Imbalanced Data

Classification

Y 95%, N 5%

Accuracy can be 100%

$\neq$  Avg. imbalanced.

$\therefore$  Solution : Stratified K-fold CV.

Fold 1

Fold 2

Fold 3.

All class are present

1, 2, 3

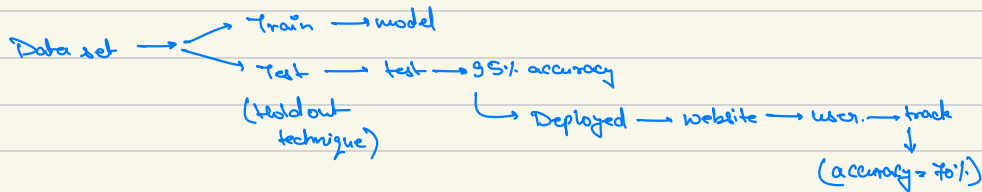
Also the ratio will be

same 2, 4, 3 respectively.

Basically ratio of data distribution is maintained.

# Data Leakage

The Problem :

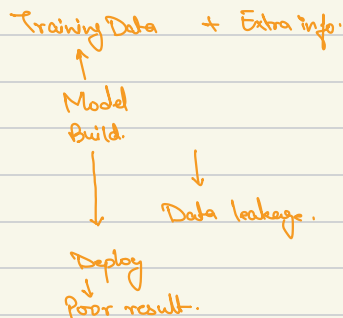


Sometimes in production, accuracy drops.

What is Data Leakage?

Data leakage, in the context of ML and DS, refers to a problem where info from outside the training dataset is used to create the model. This additional info can come in various forms, but the common characteristic is that it is information that the model wouldn't have access to when it's used for prediction in a real-world scenario.

This can lead to overly optimistic performance estimate during the training and validation, as the model has access to extra information. However, when the model is deployed at the production env., that additional info. is no longer available, and the performance of the model can drop significantly. This discrepancy is typically result of mistakes in the experiment design.



Ways in which Data Leakage can occur.

1. Target Leakage (Output col.)

It occurs when your predictors include data that will not be available at the time you make predictions.

2. Multicollinearity with target col.

3. Duplicated Data.

\* 4. Preprocessing leakage  $\rightarrow$  Train test contamination & Improper cross-validation.

5. Hyperparameter Tuning.

eg. for 1.)  
Target Leakage.

input  $\rightarrow$  (f1)  $\rightarrow$  feature  $\rightarrow$  available  $\times$ .

Credit Card Fraud Detection.

| value                    | date | website | ... | Reversed-transaction. | fraud |
|--------------------------|------|---------|-----|-----------------------|-------|
| 500                      |      |         |     | 1                     | 1     |
| normal $\rightarrow$ 400 |      |         |     | 0                     | 0     |

won't be available after training.

eg. for 4.)

Preprocessing  
 $\downarrow$   
 $n = 1000$   
 $\downarrow$   
Age  
missing = Age.mean()  
 $\downarrow$   
Train      Test

Preprocessing  
 $\downarrow$   
Standardization.  $\rightarrow$  std,  $\mu$ .  
 $\downarrow$   
Train      Test.

## How to Detect?

1. Review your features.
2. Unexpectedly High-performance.
3. Inconsistent Performance B/w Training and Unseen Data.
4. Model Interpretability.

↳ feature importance

$f_1 \rightarrow 0.13$

$f_2 \rightarrow 0.81$

↳ Data leakage.

## How to remove data leakage?

1. Understand the data and the task.
2. Careful feature selection.
- \* 3. Proper data-splitting.
- \* 4. Pre-processing inside the cross-validation loop.

```
# Incorrect way
X_normalized = normalize(X) # normalize the whole dataset
cross_val_score(model, X_normalized, y, cv=5) # perform cross-validation
```

5.

```
# Correct way
pipeline = make_pipeline(normalizer, model)
cross_val_score(pipeline, X, y, cv=5) # per
```

6. Avoid Overlapping Data.

# Hyperparameter Tuning

**Parameters:** are internal variables of a model that are learned from the data during the training process. They define the model's representation of the underlying patterns in the data.

- eg.)
- In linear regression model, the parameters are the coefficients of the predictors.
  - In NN, the parameters are the weights and biases of the nodes.
  - In DT, the parameters are the split points and split criteria at each node.

The goal of the training process is to find the optimal values for these parameters, which minimize the discrepancy b/w the model's predictions and the actual outcomes.

→ **setting knobs.**

**Hyperparameters:** In ML, hyperparameters are parameters whose values are set before the learning process begins. The parameters are not learned from the data & must be predefined. They help in controlling the learning process and can significantly influence the performance of the model.

- In NN, hyperparameters might include the learning rate, the no. of layers in the network, or the no. of nodes in each layer.
- In SVM, the regularized parameter  $C$  or the kernel type can be considered as hyperparameters.
- In a DT, the maximum depth of the tree is a hyperparameter.

The best values for hyperparameter often cannot be determined in advance, and must be found through trial and error.

Why the word 'hyper'?

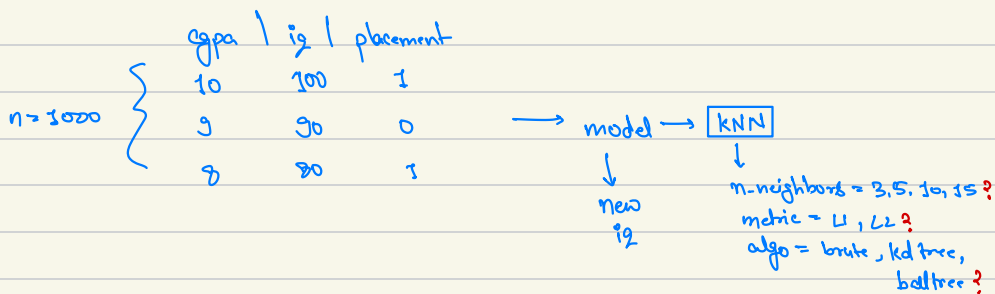
The choice of the word is primarily a naming convention to differentiate b/w the two types of values (internal parameters and guiding parameters) that influence the behaviour of a ML model. It's also a nod to the fact that the role they play is a meta one, in the sense that they control the structural aspects of the learning process itself rather than being part of the direct pattern-finding mission of the model.

Requirements :

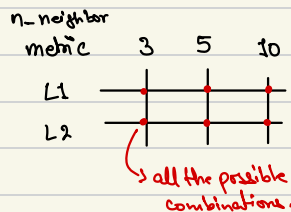
- Grid Search CV
- Randomized Search CV.

Grid Search CV :

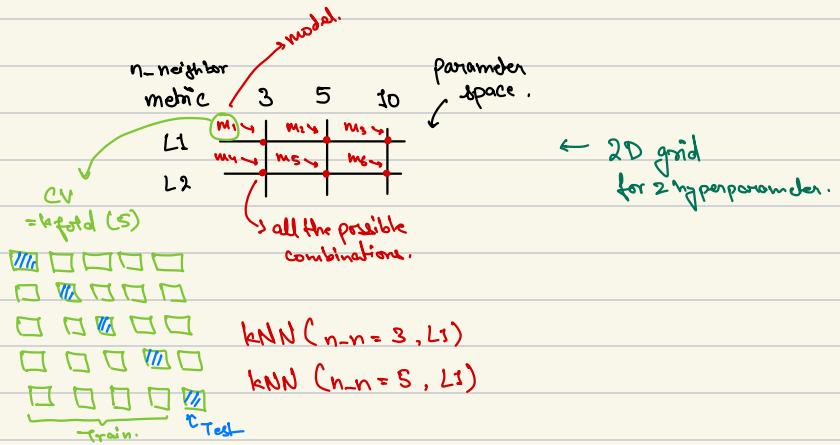
Grid Search CV refers to an algorithm that performs an exhaustive search over a specific grid of hyperparameters, using cross-validation to determine which hyperparameter combination gives the best model performance.



$n\_neighbors = 3, 5, 10$  }  $\rightarrow$  Grid  $\rightarrow$  formation  
 $metric = L1, L2$



Run the model through trial and error, search for best.



$$\text{Total} = 5 \times 6(\text{models}) = 30$$

It gives the best results, though its time taking.

3D grid for 3 parameter

40 " " 4 " "

Problem :

Like for Gradient Boosting.

15+ hyperparameters  $\rightarrow$  values  $\rightarrow$  1000s of model  $\rightarrow$  GridSearchCV Not Applicable.

+ Dataset is very big.

Training time = Years.

## Randomized Search CV :

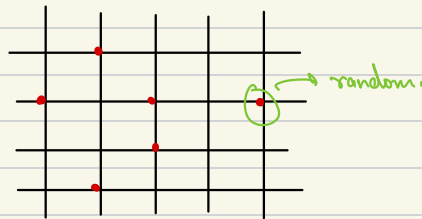
Rather than trying all the possibilities, we can try  $x$  possibilities.

Let  $x = 40, 50$ , or any number as per grid.

( $x < \text{grid\_size}$ )

$\uparrow$   
n-iter

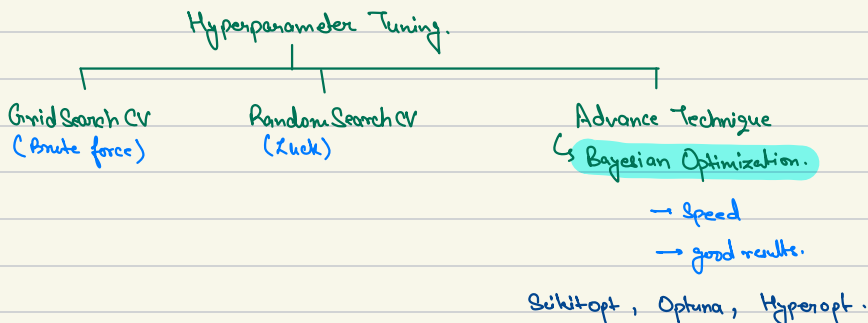
Not perfectly random, decision making somewhat depends upon the range of hyperparameter, but mostly it's random.



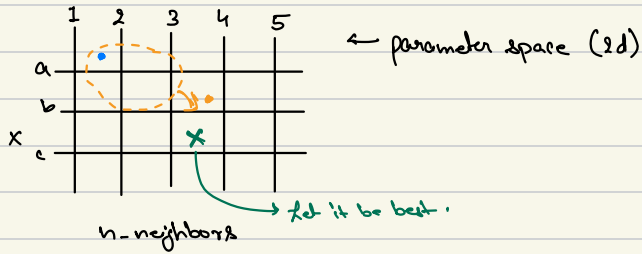
Advantages : does T.C.

Disadvantage : Might not reach the best hyperparameters.

Can this be improved ?



## Bayesian Optimization (Intuition)



## Bayesian Optimization.

